

杂项

莫队算法

离散化

整体二分

双指针

杂项

莫队算法

对于序列上 m 次询问,若从 $[l, r]$ 能 $O(1)$ 扩展到 $[l-1, r], [l+1, r], [l, r+1], [l, r-1]$ 那可以在 $O(m\sqrt{m})$ 复杂度求答案

第一关键字: 将 m 次询问分为 $\sqrt{n}$ 个区间

第二关键字: 询问区间 $[l, r]$ 中的 r

思想: 以第一个关键字的区间块排序, 再以第二关键字从小到大排序

```
1  int a[N];    //序列
2  int ans[M];  //记录每次询问答案
3  int block;   //分为多少块
4
5  struct node {
6      int l, r, id;    //分别为每次询问的左右区间,第几次的询问编号
7
8      bool operator<(const node& N) const {
9          if (l / block == N.l / block) {
10             return r < N.r;
11          } else {
12             return l < N.l;
13          }
14      }
15  } modui[M];
16
17
18  void move(int pos, int sign) {
19
20  }
21
22  void slove() {
23      blocok = sqrt(m);
24      sort(modui, modui + m);
25      int l = 1, r = 0;    //左和右指针
26      for (int i = 0; i < m; i++) {
27          node q = modui[i];
28          while (l > q.l) move(--l, 1);    //先扩大
29          while (r < q.r) move(++r, 1);
30          while (l < q.l) move(l++, -1);    //再减小
31          while (r > q.r) move(r--, -1);
32          ans[q.id] = res;
33      }
34      for (int i = 0; i < m; i++) {
35          cout << ans[i] << "\n";
36      }
37  }
```

离散化

得到排名数组

```
1 auto _ = a;
2 sort(_begin() + 1, _end());
3 _erase(unique(_begin() + 1, _end()), _end());
4 auto get_rank = [&](int x) {
5     return lower_bound(_begin() + 1, _end(), x) - _begin();
6 };
```

注：若离散化区间的话,如离散化 $[l, r]$ 得多加入 $l - 1$ 或 $r + 1$ 来防止区间端点重合

整体二分

静态第 k 小:

```
1 auto ans = [&](int n, int q, int mn, int mx) -> vector<int> {
2     vector<array<int, 2>> pos(1); //记录端点,左端点减 1, 因为要用前缀和思想
3     vector<int> ans(q + 1), cnt(q + 1), id(n + 1); //cnt 为求出当前 mid 第几大, id 为区间划分
4
5     for (int i = 1; i <= q; i++) {
6         pos.push_back({qer[i][0] - 1, i}), pos.push_back({qer[i][1], i});
7     }
8
9     iota(id.begin(), id.end(), 0ll);*
10    sort(pos.begin(), pos.end());
11
12    auto solve = [&](auto & solve, int l, int r, int al, int ar, int ql, int qr) {
13        if (l == r || al > ar || ql > qr) {
14            for (int i = ql; i <= qr; i++) {
15                ans[pos[i][1]] = l;
16            }
17            return;
18        }
19
20        int mid = l + r >> 1, am = stable_partition(id.begin() + al, id.begin() + ar + 1, [&]
21            (auto & i) {
22                return a[i] <= mid;
23            }) - id.begin() - 1; //划分区间
24
25        for (int i = ql, j = al; i <= qr; i++) { //求第 k 大 j 从 am + 1 开始
26            while (j <= am && id[j] <= pos[i][0]) { //求第 k 大到 ar 结束
27                j++;
28            }
29            if (pos[i][0] + 1 == qer[pos[i][1]][0]) { //前缀和思想快速求个数
30                cnt[pos[i][1]] -= j;
31            } else {
32                cnt[pos[i][1]] += j;
33            }
34        }
35
36        int qm = stable_partition(pos.begin() + ql, pos.begin() + qr + 1, [&](auto & x) {
37            return qer[x[1]][2] <= cnt[x[1]]; //求第 k 大时需要变成 >
38        }) - pos.begin() - 1; //划分询问
39
40        for (int i = ql; i <= qm; i++) { //更新左, 若求第 k 大时与更新右交换写法
41            cnt[pos[i][1]] = 0;
42        }
43        for (int i = qm + 1; i <= qr; i++) { //更新右
44            qer[pos[i][1]][2] -= cnt[pos[i][1]], cnt[pos[i][1]] = 0;
```

```
44     }
45
46     solve(slove, l, mid, al, am, ql, qm), solve(slove, mid + 1, r, am + 1, ar, qm + 1, qr);
47 };
48
49     solve(slove, mn, mx, 1, n, 1, 2 * q);
50
51     return ans;
52 } ();
```

双指针

维护区间信息：

若不和其他数据结构结合使用，双指针维护区间信息的简单模式就是维护具有一定单调性，新增和删去一个元素都很方便处理的信息

- 维护区间 gcd，因为 gcd 暴力维护区间 gcd 不会超过 $\log$